

My Project

Generated by Doxygen 1.15.0

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 NormalisedConditionalRandomField.py File Reference	3
2.1.1 Detailed Description	4
2.1.2 Function Documentation	4
2.1.2.1 apply_dense_crf()	4
2.1.2.2 calculate_dice_coefficient()	5
2.1.2.3 evaluate_segmentation()	5
2.1.2.4 extract_features_with_coords()	6
2.1.2.5 get_probability_map()	6
2.1.2.6 main()	7
2.1.2.7 map_labels_back()	7
2.1.2.8 predict_with_crf()	8
2.1.2.9 remap_mask_labels()	8
2.1.2.10 resize_image()	9
2.1.2.11 resize_mask()	9
2.1.2.12 save_comparison_metrics_as_image()	10
2.1.2.13 save_summary_metrics_as_image()	10
2.1.2.14 train_naive_bayes_memory_efficient()	11
2.1.2.15 visualize_crf_comparison()	11
Index	13

Chapter 1

File Index

1.1 File List

Here is a list of all documented files with brief descriptions:

[NormalisedConditionalRandomField.py](#)

Optic disc and cup segmentation using Naive Bayes classifier with Dense CRF refinement and
normalized features

3

Chapter 2

File Documentation

2.1 NormalisedConditionalRandomField.py File Reference

Optic disc and cup segmentation using Naive Bayes classifier with Dense CRF refinement and normalized features.

Functions

- [NormalisedConditionalRandomField.resize_image](#) (image, max_size=512)
Resize regular fundus image to reduce memory usage.
- [NormalisedConditionalRandomField.resize_mask](#) (mask, max_size=512)
Resize segmentation mask using nearest neighbor interpolation.
- [NormalisedConditionalRandomField.remap_mask_labels](#) (mask)
Remap mask pixel values to standardized label values.
- [NormalisedConditionalRandomField.extract_features_with_coords](#) (image, normalize_coords=True, use_both_colorspaces=True, normalize_colors=True)
Extract color and spatial features from fundus image with optional normalization.
- [NormalisedConditionalRandomField.train_naive_bayes_memory_efficient](#) (fundus_files, mask_files, use_both_colorspaces=True, batch_size=5, max_size=512)
Train Naive Bayes classifier using batch processing and incremental learning.
- [NormalisedConditionalRandomField.get_probability_map](#) (clf, scaler, image, use_both_colorspaces=True)
Generate probability map from Naive Bayes classifier predictions.
- [NormalisedConditionalRandomField.apply_dense_crf](#) (image, probabilities, n_iters=5, sxy_gaussian=3, compat_gaussian=3, sxy_bilateral=80, srgb_bilateral=13, compat_bilateral=10)
Apply Dense Conditional Random Field refinement to probability map.
- [NormalisedConditionalRandomField.map_labels_back](#) (crf_output, label_mapping)
Map CRF integer labels back to original mask values.
- [NormalisedConditionalRandomField.predict_with_crf](#) (clf, scaler, test_image, use_both_colorspaces=True, apply_crf=True, crf_params=None)
Predict segmentation with optional CRF refinement.
- [NormalisedConditionalRandomField.calculate_dice_coefficient](#) (pred, gt, label)
Calculate Dice coefficient for a specific segmentation class.
- [NormalisedConditionalRandomField.evaluate_segmentation](#) (pred_mask, gt_mask)
Evaluate segmentation performance using multiple metrics.
- [NormalisedConditionalRandomField.save_comparison_metrics_as_image](#) (metrics_nb, metrics_crf, image_name, save_path)
Save comparison table of Naive Bayes and CRF metrics as image.

- [NormalisedConditionalRandomField.save_summary_metrics_as_image](#) (metrics_nb_list, metrics_crf_list, save_path)
Save overall summary metrics with mean and standard deviation statistics.
- [NormalisedConditionalRandomField.visualize_crf_comparison](#) (original, naive_mask, crf_mask, gt_mask=None, save_path=None)
Visualize comparison between Naive Bayes and CRF segmentation results.
- [NormalisedConditionalRandomField.main](#) ()
Memory-efficient main execution pipeline with normalized features and CRF refinement.

2.1.1 Detailed Description

Optic disc and cup segmentation using Naive Bayes classifier with Dense CRF refinement and normalized features.

This module implements a memory-efficient pipeline for retinal image segmentation that combines Gaussian Naive Bayes classification with normalized color features and Dense Conditional Random Field (CRF) post-processing to segment optic disc, cup, and background regions from fundus images. Includes comprehensive metrics comparison with mean and standard deviation reporting.

Author

Rishab Ramesh Nair

Date

November 2, 2025

2.1.2 Function Documentation

2.1.2.1 apply_dense_crf()

```
NormalisedConditionalRandomField.apply_dense_crf (
    image,
    probabilities,
    n_iters = 5,
    sxy_gaussian = 3,
    compat_gaussian = 3,
    sxy_bilateral = 80,
    srgb_bilateral = 13,
    compat_bilateral = 10)
```

Apply Dense Conditional Random Field refinement to probability map.

Refines segmentation predictions using Dense CRF with Gaussian and bilateral pairwise potentials to enforce spatial smoothness and edge-aware consistency.

Parameters

<i>image</i>	Input RGB image for bilateral filtering.
<i>probabilities</i>	Class probability array of shape (n_classes, height, width).

<i>n_iters</i>	Number of CRF inference iterations. Default is 5.
<i>sxy_gaussian</i>	Gaussian kernel spatial standard deviation. Default is 3.
<i>compat_gaussian</i>	Gaussian kernel compatibility weight. Default is 3.
<i>sxy_bilateral</i>	Bilateral kernel spatial standard deviation. Default is 80.
<i>srgb_bilateral</i>	Bilateral kernel color standard deviation. Default is 13.
<i>compat_bilateral</i>	Bilateral kernel compatibility weight. Default is 10.

Returns

Refined segmentation map of shape (height, width) with integer class labels.

```
Apply Dense CRF refinement
```

2.1.2.2 calculate_dice_coefficient()

```
NormalisedConditionalRandomField.calculate_dice_coefficient (
    pred,
    gt,
    label)
```

Calculate Dice coefficient for a specific segmentation class.

Computes the Dice similarity coefficient (F1 score) between predicted and ground truth segmentation for a single class label. Formula: $2 * |X \cap Y| / (|X| + |Y|)$

Parameters

<i>pred</i>	Predicted segmentation mask.
<i>gt</i>	Ground truth segmentation mask.
<i>label</i>	Class label value to evaluate (0, 128, or 255).

Returns

Dice coefficient value in range [0, 1] where 1 indicates perfect overlap.

```
Calculate Dice coefficient for a specific class
```

2.1.2.3 evaluate_segmentation()

```
NormalisedConditionalRandomField.evaluate_segmentation (
    pred_mask,
    gt_mask)
```

Evaluate segmentation performance using multiple metrics.

Computes comprehensive evaluation metrics including overall pixel accuracy and per-class Dice coefficients for all three classes (disc, cup, background).

Parameters

<i>pred_mask</i>	Predicted segmentation mask.
<i>gt_mask</i>	Ground truth segmentation mask.

Returns

Dictionary containing the following metrics:

- 'accuracy': Overall pixel accuracy
- 'dice_disc': Dice coefficient for disc class (0)
- 'dice_cup': Dice coefficient for cup class (128)
- 'dice_background': Dice coefficient for background class (255)
- 'dice_mean': Mean Dice across all classes

Evaluate segmentation performance

2.1.2.4 extract_features_with_coords()

```
NormalisedConditionalRandomField.extract_features_with_coords (
    image,
    normalize_coords = True,
    use_both_colorspaces = True,
    normalize_colors = True)
```

Extract color and spatial features from fundus image with optional normalization.

Extracts pixel-wise features including RGB and optionally HSV color information along with normalized or absolute spatial coordinates. Supports feature normalization to [0,1] range for improved numerical stability.

Parameters

<i>image</i>	Input fundus image (BGR format).
<i>normalize_coords</i>	If True, normalize coordinates to [0,1] range. Default is True.
<i>use_both_colorspaces</i>	If True, extract both RGB and HSV features. Default is True.
<i>normalize_colors</i>	If True, normalize color values to [0,1] range. Default is True.

Returns

Feature array of shape (height*width, n_features) where n_features is 8 (RGB + HSV + x + y) if use_both_colorspaces=True, or 5 (RGB + x + y) if use_both_colorspaces=False.

Extract color and spatial features from fundus image

2.1.2.5 get_probability_map()

```
NormalisedConditionalRandomField.get_probability_map (
    clf,
    scaler,
    image,
    use_both_colorspaces = True)
```

Generate probability map from Naive Bayes classifier predictions.

Computes per-pixel class probabilities for all segmentation classes using the trained Naive Bayes classifier with normalized features.

Parameters

<i>clf</i>	Trained GaussianNB classifier.
<i>scaler</i>	Fitted StandardScaler for feature normalization.
<i>image</i>	Input fundus image (BGR format).
<i>use_both_colorspaces</i>	If True, use both RGB and HSV features. Default is True.

Returns

Probability array of shape (n_classes, height, width) with values in [0,1].

Get probability map from Naive Bayes classifier

2.1.2.6 main()

NormalisedConditionalRandomField.main ()

Memory-efficient main execution pipeline with normalized features and CRF refinement.

Executes the complete segmentation workflow including:

- Configuration of data directories and processing parameters
- Loading and preprocessing training data with memory-efficient batching
- Training Gaussian Naive Bayes classifier with normalized color features
- Testing on validation images with ground truth evaluation
- Applying Dense CRF post-processing for segmentation refinement
- Computing comprehensive metrics with mean and standard deviation statistics
- Saving individual and summary visualizations and metric tables

The pipeline uses memory optimization techniques including image resizing, batch processing, normalized features, and explicit garbage collection.

Returns

None. All outputs (predictions, metrics, visualizations) are saved to OUTPUT_DIR.

Memory-efficient main execution pipeline with CRF

2.1.2.7 map_labels_back()

NormalisedConditionalRandomField.map_labels_back (
 crf_output,
 label_mapping)

Map CRF integer labels back to original mask values.

Converts class indices from CRF output back to the original grayscale mask values (0, 128, 255).

Parameters

<i>crf_output</i>	CRF output with integer class labels.
<i>label_mapping</i>	Dictionary mapping integer labels to original mask values.

Returns

Final segmentation mask with original label values.

Map CRF integer labels back to original mask values

2.1.2.8 predict_with_crf()

```
NormalisedConditionalRandomField.predict_with_crf (
    clf,
    scaler,
    test_image,
    use_both_colorspaces = True,
    apply_crf = True,
    crf_params = None)
```

Predict segmentation with optional CRF refinement.

Generates segmentation predictions using Naive Bayes classifier with normalized features and optionally applies Dense CRF post-processing for refinement.

Parameters

<i>clf</i>	Trained GaussianNB classifier.
<i>scaler</i>	Fitted StandardScaler for feature normalization.
<i>test_image</i>	Input test fundus image (BGR format).
<i>use_both_colorspaces</i>	If True, use both RGB and HSV features. Default is True.
<i>apply_crf</i>	If True, apply CRF refinement. Default is True.
<i>crf_params</i>	Dictionary of CRF parameters. If None, uses default values.

Returns

Tuple of (final_mask, naive_bayes_mask) where final_mask is the CRF-refined result (or Naive Bayes result if apply_crf=False) and naive_bayes_mask is the unrefined classifier output.

Predict segmentation with optional CRF refinement

2.1.2.9 remap_mask_labels()

```
NormalisedConditionalRandomField.remap_mask_labels (
    mask)
```

Remap mask pixel values to standardized label values.

Converts grayscale mask values to exactly three discrete labels based on intensity thresholds:

- Values 0-63 map to 0 (disc)
- Values 64-191 map to 128 (cup)
- Values 192-255 map to 255 (background)

Parameters

<i>mask</i>	Input grayscale mask with arbitrary intensity values.
-------------	---

Returns

Remapped mask with values in {0, 128, 255}.

Remap mask values to exactly 0, 128, or 255

2.1.2.10 `resize_image()`

```
NormalisedConditionalRandomField.resize_image (  
    image,  
    max_size = 512)
```

Resize regular fundus image to reduce memory usage.

Proportionally resizes an image so that its largest dimension does not exceed the specified maximum size while maintaining aspect ratio using area interpolation.

Parameters

<i>image</i>	Input image array (BGR format).
<i>max_size</i>	Maximum dimension size in pixels. Default is 512.

Returns

Resized image or original image if already within size limit.

Resize regular image (fundus) to reduce memory usage

2.1.2.11 `resize_mask()`

```
NormalisedConditionalRandomField.resize_mask (  
    mask,  
    max_size = 512)
```

Resize segmentation mask using nearest neighbor interpolation.

Resizes a mask image while preserving discrete label values by using nearest neighbor interpolation to avoid creating intermediate values.

Parameters

<i>mask</i>	Input mask array with discrete labels.
<i>max_size</i>	Maximum dimension size in pixels. Default is 512.

Returns

Resized mask or original mask if already within size limit.

Resize mask using nearest neighbor to preserve discrete labels

2.1.2.12 `save_comparison_metrics_as_image()`

```
NormalisedConditionalRandomField.save_comparison_metrics_as_image (
    metrics_nb,
    metrics_crf,
    image_name,
    save_path)
```

Save comparison table of Naive Bayes and CRF metrics as image.

Creates and saves a formatted table comparing segmentation metrics between Naive Bayes and CRF-refined predictions for a single test image, with color-coded improvement indicators.

Parameters

<i>metrics_nb</i>	Dictionary of Naive Bayes metrics.
<i>metrics_crf</i>	Dictionary of CRF-refined metrics.
<i>image_name</i>	Name of the test image.
<i>save_path</i>	File path where the metrics comparison image will be saved.

Returns

None. Saves image to disk.

Save comparison of Naive Bayes and CRF metrics as an image

2.1.2.13 `save_summary_metrics_as_image()`

```
NormalisedConditionalRandomField.save_summary_metrics_as_image (
    metrics_nb_list,
    metrics_crf_list,
    save_path)
```

Save overall summary metrics with mean and standard deviation statistics.

Computes and saves a comprehensive table showing mean and standard deviation for all metrics across multiple test images, comparing Naive Bayes and CRF performance with improvement indicators.

Parameters

<i>metrics_nb_list</i>	List of Naive Bayes metrics dictionaries for all test images.
<i>metrics_crf_list</i>	List of CRF metrics dictionaries for all test images.
<i>save_path</i>	File path where the summary metrics image will be saved.

Returns

None. Saves image to disk or returns early if `metrics_crf_list` is empty.

Save overall summary metrics with mean and std deviation

2.1.2.14 train_naive_bayes_memory_efficient()

```
NormalisedConditionalRandomField.train_naive_bayes_memory_efficient (
    fundus_files,
    mask_files,
    use_both_colorspaces = True,
    batch_size = 5,
    max_size = 512)
```

Train Naive Bayes classifier using batch processing and incremental learning.

Trains a Gaussian Naive Bayes classifier on fundus images and their segmentation masks using incremental learning with batch processing to reduce memory usage. Uses `partial_fit` for online learning across batches with normalized features.

Parameters

<i>fundus_files</i>	List of file paths to training fundus images.
<i>mask_files</i>	List of file paths to corresponding segmentation masks.
<i>use_both_colorspaces</i>	If True, use both RGB and HSV color spaces. Default is True.
<i>batch_size</i>	Number of images to process per batch. Default is 5.
<i>max_size</i>	Maximum image dimension for resizing. Default is 512.

Returns

Tuple of (trained_classifier, fitted_scaler) where:

- trained_classifier is a GaussianNB object trained on all batches
- fitted_scaler is a StandardScaler for feature normalization

Memory-efficient training with batching

2.1.2.15 visualize_crf_comparison()

```
NormalisedConditionalRandomField.visualize_crf_comparison (
    original,
    naive_mask,
    crf_mask,
    gt_mask = None,
    save_path = None)
```

Visualize comparison between Naive Bayes and CRF segmentation results.

Creates a side-by-side visualization comparing original image, ground truth (if available), Naive Bayes prediction, and CRF-refined prediction.

Parameters

<i>original</i>	Original fundus image (BGR format).
-----------------	-------------------------------------

<i>naive_mask</i>	Naive Bayes segmentation mask.
<i>crf_mask</i>	CRF-refined segmentation mask.
<i>gt_mask</i>	Ground truth mask. If None, only shows 3 panels. Default is None.
<i>save_path</i>	File path to save visualization. If None, displays instead. Default is None.

Returns

None. Either saves or displays the visualization.

Visualize comparison between Naive Bayes and CRF refinement

Index

`apply_dense_crf`
 [NormalisedConditionalRandomField.py, 4](#)

`calculate_dice_coefficient`
 [NormalisedConditionalRandomField.py, 5](#)

`evaluate_segmentation`
 [NormalisedConditionalRandomField.py, 5](#)

`extract_features_with_coords`
 [NormalisedConditionalRandomField.py, 6](#)

`get_probability_map`
 [NormalisedConditionalRandomField.py, 6](#)

`main`
 [NormalisedConditionalRandomField.py, 7](#)

`map_labels_back`
 [NormalisedConditionalRandomField.py, 7](#)

[NormalisedConditionalRandomField.py, 3](#)
 [apply_dense_crf, 4](#)
 [calculate_dice_coefficient, 5](#)
 [evaluate_segmentation, 5](#)
 [extract_features_with_coords, 6](#)
 [get_probability_map, 6](#)
 [main, 7](#)
 [map_labels_back, 7](#)
 [predict_with_crf, 8](#)
 [remap_mask_labels, 8](#)
 [resize_image, 9](#)
 [resize_mask, 9](#)
 [save_comparison_metrics_as_image, 9](#)
 [save_summary_metrics_as_image, 10](#)
 [train_naive_bayes_memory_efficient, 10](#)
 [visualize_crf_comparison, 11](#)

`predict_with_crf`
 [NormalisedConditionalRandomField.py, 8](#)

`remap_mask_labels`
 [NormalisedConditionalRandomField.py, 8](#)

`resize_image`
 [NormalisedConditionalRandomField.py, 9](#)

`resize_mask`
 [NormalisedConditionalRandomField.py, 9](#)

`save_comparison_metrics_as_image`
 [NormalisedConditionalRandomField.py, 9](#)

`save_summary_metrics_as_image`
 [NormalisedConditionalRandomField.py, 10](#)

`train_naive_bayes_memory_efficient`
 [NormalisedConditionalRandomField.py, 10](#)

`visualize_crf_comparison`
 [NormalisedConditionalRandomField.py, 11](#)